Relatório Técnico: Implementação do Algoritmo S-AES em MIPS Assembly

Lucas Xavier Pairé Miguel Brondani

Universidade Federal de Santa Maria (UFSM)
Organização de Computadores
Professor: Giovani Baratto

1 Introdução

O desenvolvimento de programas em linguagem de montagem permite entender a relação direta entre as instruções de máquina, a organização do processador e o gerenciamento de memória. Neste trabalho, implementamos um sistema de criptografia e descriptografia de arquivos para a arquitetura MIPS.

A definição do algoritmo a ser implementado foi o Simplified Advanced Encryption Standard (S-AES).

O sistema construído opera por meio de uma interface no terminal, oferecendo operações de cifra e decifra de arquivos de texto armazenados em disco. O projeto foi organizado em módulos independentes que respeitam a convenção de chamadas do processador MIPS.

2 Objetivos

O objetivo geral do trabalho consiste em projetar e implementar um programa em linguagem Assembly MIPS capaz de cifrar e decifrar arquivos de texto utilizando o algoritmo criptográfico S-AES, validando a integridade das informações processadas no simulador MARS.

Os objetivos específicos definidos para o projeto são:

1. Implementar o algoritmo do S-AES, contendo a expansão da chave em subchaves de rodada e as rotinas de transformação direta e inversa.

2. Implementar lógica de entrada e saída de arquivos com leitura e escrita em blocos de tamanho fixo, permitindo o processamento de arquivos de diferentes tamanhos sem sobrecarregar a memória.

3. Desenvolver a interface com o usuário no terminal, gerenciando a navegação por menus, a captura de caminhos de arquivos e a leitura da senha de acesso.

4. Implementar uma lógica para converter a senha textual de entrada na chave numérica de 16 bits.

5. Tratar arquivos com quantidade ímpar de bytes através de preenchimento, garantindo blocos de 16 bits.

6. Validar o funcionamento das rotinas por meio de vetores de teste clássicos da literatura e através de arquivos de teste.

3 Referencial Teórico

O algoritmo S-AES foi desenvolvido originalmente por Edward Schaefer em 1996 e documentado por William Stallings como uma ferramenta para o ensino de criptografia simétrica. Trata-se de uma versão simplificada do algoritmo AES padrão, preservando sua estrutura algébrica e a sequência de operações, porém operando com blocos de dados de 16 bits e chaves de 16 bits.

3.1 Estrutura do Estado e Representação dos Dados

No S-AES, o bloco de 16 bits é tratado como uma matriz de estado de duas linhas e duas colunas, na qual cada elemento é um nibble de 4 bits. A disposição dos nibbles dentro da palavra de 16 bits segue a ordem em que o nibble mais significativo ocupa a posição superior esquerda da matriz, seguido pelo nibble inferior esquerdo, superior direito e inferior direito.

3.2 Expansão de Chave

A chave de entrada possui 16 bits, representada por duas palavras de 8 bits denominadas w_0 e w_1. O processo de expansão gera quatro palavras adicionais de 8 bits (w_2, w_3, w_4 e w_5), permitindo a formação de três subchaves de 16 bits:

1. Subchave 0 (Key_0): formada pela junção de w_0 e w_1.
2. Subchave 1 (Key_1): formada pela junção de w_2 e w_3.
3. Subchave 2 (Key_2): formada pela junção de w_4 e w_5.

Para gerar as palavras de índice par (w_2 e w_4), utiliza-se a função auxiliar g, que executa três passos sobre a palavra anterior:

1. Rotação de nibbles (RotNib): troca a posição dos dois nibbles de 4 bits.
2. Substituição de nibbles (SubNib): aplica a caixa de substituição (SBox) sobre cada nibble.
3. Operação lógica XOR com a constante de rodada RCON. A constante para a primeira rodada é `0x80` e para a segunda rodada é `0x30`.

As palavras seguintes são calculadas por operações XOR entre as palavras já existentes, segundo a regra matemática do algoritmo.

3.3 Transformações da Cifra

O processo de encriptação é composto por uma rodada preliminar e duas rodadas principais:

1. Rodada Inicial: realiza a adição da chave de rodada (AddRoundKey) combinando o bloco original com a subchave Key_0 por meio de XOR.

2. Rodada 1: executa a substituição de nibbles (SubNibbles) mapeando cada valor de 4 bits pela SBox, o deslocamento de linhas (ShiftRows) que permuta os nibbles da segunda linha da matriz de estado, a mistura de colunas (MixColumns) e a adição da subchave Key_1.

3. Rodada 2: repete as operações de substituição de nibbles e deslocamento de linhas, finalizando com a adição da subchave Key_2, omitindo a mistura de colunas.

3.4 Aritmética no Corpo Finito $GF(16)$

A transformação de mistura de colunas multiplica cada coluna da matriz de estado por uma matriz constante fixa. As operações de adição correspondem ao XOR binário, enquanto as multiplicações ocorrem no corpo finito de Galois $GF(16)$, utilizando o polinômio irredutível $x^4 + x + 1$. Elementos desse corpo assumem valores de 0 até 15.

3.5 Transformações da Decifra

A descriptografia percorre o caminho inverso da cifra, aplicando as operações matemáticas simétricas:

1. Rodada Inicial: adição da subchave Key_2 sobre o bloco cifrado.

2. Rodada 1: inversão do deslocamento de linhas, substituição pela SBox invertida, adição da subchave Key_1 e aplicação da mistura de colunas inversa com a matriz de coeficientes apropriada no corpo finito.

3. Rodada 2: inversão do deslocamento de linhas, substituição pela SBox invertida e adição da subchave Key_0.

4 Metodologia

A construção do trabalho foi orientada pela separação de responsabilidades em módulos.

4.1 Organização dos Módulos

O código do projeto foi estruturado em cinco arquivos principais:

1. `main.asm`: contém o laço principal do programa, a exibição do menu de opções no terminal, a captura de strings e o gerenciamento do fluxo de execução.

2. `file_io.asm`: implementa o processamento de arquivos em disco, controlando abertura, leitura sequencial, iteração de blocos e escrita dos dados.

3. `saes.asm`: implementa o algoritmo criptográfico, contendo a expansão de chaves, as transformações atômicas e as funções completas de cifra e decifra de blocos.

4. `saes_tables.asm`: define as tabelas estáticas na memória de dados, incluindo a caixa de substituição direta, a caixa de substituição inversa e as matrizes de multiplicação no corpo finito.

5. `utils.asm`: reúne funções utilitárias para sanitização de strings e derivação da chave a partir da senha digitada.

4.2 Multiplicação em $GF(16)$ com Tabelas Estáticas

A multiplicação polinomial com redução algébrica em tempo de execução no MIPS exigiria laços de deslocamento, testes condicionais e divisões, o que tornaria o código complexo e propenso a erros. Para maximizar a eficiência e a clareza, implementamos a aritmética de mistura de colunas utilizando tabelas previamente calculadas na seção de dados.

Foram alocadas três tabelas de 16 bytes:

1. `tab_mul_2`: armazena os produtos de cada valor de 0 a 15 por 2 no corpo finito.

2. `tab_mul_4`: armazena os produtos de cada valor de 0 a 15 por 4 no corpo finito.

3. `tab_mul_9`: armazena os produtos de cada valor de 0 a 15 por 9 no corpo finito.

Dessa forma, qualquer multiplicação em $GF(16)$ é realizada em apenas duas instruções MIPS: uma soma para indexar a tabela e uma leitura de byte da memória.

4.3 Processamento de Arquivos com Buffers

O programa manipula arquivos, de forma que a leitura e a escrita não carregam o arquivo inteiro de uma só vez. O processamento ocorre em partes utilizando dois buffers de 512 bytes alocados estaticamente na memória: um buffer de entrada e um buffer de saída.

O procedimento processa os arquivos em etapas sucessivas:

1. Abre o arquivo de entrada com a chamada de sistema 13 em modo somente leitura.

2. Abre o arquivo de saída com a chamada de sistema 13 em modo de escrita com criação e truncamento.

3. Lê até 512 bytes no buffer de entrada com a chamada de sistema 14.

4. Percorre o buffer de dois em dois bytes (16 bits por bloco), invocando a função de cifra ou decifra para cada bloco.

5. Grava os bytes resultantes no buffer de saída e descarrega no disco com a chamada de sistema 15.

6. Repete o ciclo até que a chamada de leitura retorne zero bytes lidos, fechando os descritores de arquivo com a chamada de sistema 16.

4.4 Tratamento de Preenchimento para Arquivos Ímpares

Como cada bloco do S-AES possui exatamente 16 bits (2 bytes), arquivos com quantidade ímpar de bytes deixam um byte isolado no final do processamento. O arquivo de teste `texto1.txt`, por exemplo, possui exatamente 111 bytes.

Durante a criptografia, a rotina de leitura verifica a quantidade de bytes obtida no último bloco lido. Ao identificar que o tamanho é ímpar, o programa insere automaticamente o byte `0x00` na posição seguinte do buffer, ajustando o contador para o próximo número par. Isso permite que o bloco final seja cifrado regularmente como uma palavra completa de 16 bits.

4.5 Derivação da Chave Criptográfica

Na descrição do trabalho é definido que os testes devem ser realizados com a senha de acesso `ELC1011`. Essa string possui 7 caracteres ASCII, totalizando 56 bits, ao passo que o algoritmo S-AES exige estritamente uma chave de 16 bits (2 bytes).

Foi implementada uma solução `deriva_chave`:

1. O procedimento percorre todos os caracteres válidos da string no terminal.
2. Os caracteres situados em índices pares são acumulados por meio de operações XOR no byte mais significativo da chave.
3. Os caracteres situados em índices ímpares são acumulados por meio de operações XOR no byte menos significativo da chave.

Ao aplicar essa regra sobre a palavra `ELC1011`, todos os 7 caracteres participam da formação do segredo criptográfico, gerando a chave de 16 bits com valor hexadecimal `0x074C`.

4.6 Convenção de Chamadas e Uso da Pilha

O código segue as boas práticas. Funções que realizam apenas operações aritméticas e lógicas utilizam exclusivamente os registradores temporários `$t0` até `$t9`, dispensando o uso da pilha.

Funções que realizam chamadas a outros procedimentos salvam previamente o registrador de endereço de retorno `$ra` e os registradores protegidos `$s0` até `$s7` no topo da pilha, restaurando os valores originais antes da instrução de retorno `jr $ra`.

5 Experimentos e Resultados

5.1 Teste Unitário com Vetor Padrão da Literatura

Para comprovar que os códigos do S-AES reproduzem o comportamento teórico, criamos o teste unitário `test_saes_vectors.asm` com base nos vetores oficiais publicados por Steven Gordon e William Stallings.

Os parâmetros utilizados no teste foram:

1. Texto claro original: `0xD728`

2. Chave de teste: `0x4AF5`

A execução do teste unitário no simulador MARS apresentou as seguintes saídas intermediárias e finais:

1. Expansão de chaves gerou Key_0 igual a `0x4AF5`, Key_1 igual a `0xDD28` e Key_2 igual a `0x87AF`, exatamente idênticas aos valores da literatura.

2. A criptografia de `0xD728` produziu o texto cifrado `0x24EC`.

3. A descriptografia de `0x24EC` retornou o texto claro original `0xD728`.

5.2 Teste Integrado com texto1.txt

O primeiro arquivo fornecido para validação foi `texto1.txt`, com tamanho original de 111 bytes. O programa foi executado interativamente com as seguintes ações:

1. Criptografia de `texto1.txt` utilizando a chave `ELC1011`, gerando o arquivo `secreto1.aes`. O arquivo gerado apresentou 112 bytes, refletindo o preenchimento automático de um byte nulo.

2. Descriptografia de `secreto1.aes` utilizando a mesma chave `ELC1011`, gerando o arquivo `teste1.txt`.

A comparação de quantidade de bytes entre `texto1.txt` e `teste1.txt` confirmou que todo o conteúdo textual original foi recuperado sem nenhuma perda.

5.3 Teste Integrado com texto2.txt

O segundo arquivo de validação foi `texto2.txt`, composto por 238 bytes.
O procedimento realizado seguiu os mesmos passos:

1. Criptografia de `texto2.txt` com a chave `ELC1011`, gerando o arquivo `secreto2.aes` com 238 bytes.

2. Descriptografia de `secreto2.aes` com a chave `ELC1011`, gerando o arquivo `teste2.txt` com 238 bytes.

A comparação de quantidade de bytes entre `texto2.txt` e `teste2.txt` confirmou que todo o conteúdo textual original foi recuperado sem nenhuma perda.

6 Discussão

Os experimentos realizados demonstraram que a utilização de tabelas estáticas de multiplicação em $GF(16)$ simplificou o código em Assembly. Uma rotina de multiplicação algorítmica em MIPS precisaria de vários desvios condicionais e cálculos bit a bit, aumentando a probabilidade de falhas lógicas e elevando o tempo de execução no simulador. A abordagem com tabelas indexadas transformou operações algébricas em acessos diretos à memória, tornando a leitura do código evidente e facilitando a depuração.

A estratégia de derivação da chave mostrou-se muito boa, para conseguirmos ter uma chave respeitando o limite de 16 bits e usando todos os caracteres.

O preenchimento com zero permitiu cifrar arquivos de tamanho ímpar sem complicar. Como os arquivos fornecidos eram textos terminados em caractere de fim de linha, a presença de um byte nulo ao término do arquivo decifrado preserva o conteúdo sem afetar o texto.

A modularização em cinco arquivos também ajudou no desenvolvimento. Isolando as rotinas criptográficas em `saes.asm` e o acesso ao disco em `file_io.asm`, foi possível validar o algoritmo matemático com vetores unitários antes mesmo de conectar as chamadas de sistema para manipulação de arquivos.

7 Conclusões

A implementação do sistema criptográfico SAES atingiu com sucesso todos os objetivos propostos.

O programa apresenta interface para o usuário, processa arquivos em blocos de memória e executa a cifragem e decifragem.